Dewlock Copilot - Command Guide

Welcome to the Dewlock copilot. This guide covers the exact commands you can use, what the copilot will show you, and the honest status of each feature.

Getting Started

1. **Connect your wallet** at https://dewlock.vercel.app/app
2. **Type a goal** in plain language in the chat box (e.g., "swap 10 SUI to USDC", "send 1 SUI to alice.sui")
3. **Review the preview card** - see exactly what will move, where it goes, and the estimated cost
4. **Sign in your wallet** - the transaction bytes you sign are the exact bytes the Guardian verified
5. **View the receipt** - an immutable record is written to Walrus; you earn XP and badges

Safety guarantee: Every transaction passes through a fail-closed Guardian. An unsafe intent is BLOCKED before you sign - no exceptions.

Commands by Feature

Swap / Sell

Status: WORKS
End-user commands: - `"swap 10 SUI to USDC"` - `"swap 100 USDC to afSUI"` - `"sell 50 SUI"`

What you see: A picker card showing real-time quotes from Cetus Aggregator and Aftermath; you choose the venue. The preview shows the exact output token, estimated USD value, and slippage protection (default 5%, configurable). The copilot will refuse a swap if the output is worth significantly less than the input (bad rate protection).

Tech: Best-execution routing across Cetus and Aftermath with source-aware min-output re-derivation. Every swap re-derives the minimum output independently - the Guardian catches sandwich/stale-price attacks.

Send / Transfer

Status: WORKS
End-user commands: - `"send 1 SUI to alice.sui"` - `"send 2 SUI to roast2026wc"` - `"send 100 USDC to 0x1234…"` - `"@alice 5 SUI"`

What you see: If you type a name (e.g., `alice.sui` or `alice`), the copilot resolves it - either via SuiNS or your saved friend book. If the name matches multiple contacts, you'll get a picker card. The preview shows the recipient address, the amount, and a homoglyph-guard alert if the name looks suspiciously similar to a known contact (spoofing protection).

Tech: SuiNS resolution + per-wallet friend address book (encrypted as a Walrus blob). The Guardian blocks homoglyph lookalikes (e.g., `888-l.sui` vs `888-1.sui`). If a recipient is inferred from memory or a pool, you'll see a confirm gate - anti-injection safety.

Lending - Deposit & Repay (Health-Improving)

Status: WORKS
End-user commands: - `"deposit 50 SUI to NAVI"` - `"deposit 100 USDC to Suilend"` - `"repay 10 USDC on NAVI"` - `"repay 5 SUI on Suilend"`

What you see: A protocol picker (if ambiguous) and a preview showing your current health factor and what it will be after the action. The copilot will show you the live lending rate and your position details.

Tech: Integrations with NAVI (liquidation prevention + official health factor simulation) and Suilend (official APY reads). Every deposit/repay is health-improving only - no risky moves.

Lending - Borrow & Withdraw (Health-Reducing)

Status: WORKS - Guardian blocks unsafe borrows
End-user commands: - `"borrow 50 USDC on NAVI"` - `"borrow 10 ETH on NAVI"` - `"withdraw 5 SUI from NAVI"`

What you see: The preview shows your current health factor and the projected health factor **after** the borrow/withdraw. If the projected health factor would fall below the safe threshold (1.6 by default), the Guardian **blocks** the action before you sign - it will show you a card explaining why.

Tech: NAVI-only (Suilend does not support borrow/withdraw). The Guardian calls NAVI's `getSimulatedHealthFactor` to verify the health factor independently - it is not a hand-rolled formula. A borrow is also capped per transaction via the server's per-tx USD cap.

Liquid Staking - afSUI (Aftermath)

Status: WORKS - mainnet-verified
End-user commands: - `"stake 10 SUI"` - `"stake 50 SUI to afSUI"` - `"unstake 5 afSUI"`

What you see: A staking picker card showing live APY from Aftermath and the current afSUI/SUI exchange rate. On stake, you mint afSUI; on unstake, you get instant atomic redemption (no epoch delay). The preview shows the expected output.

Tech: Built on Aftermath's `staked_sui_vault`. The Guardian enforces coin-type provenance (scam-clone afSUI is blocked) and prices the afSUI using an independent floor formula (not Aftermath's own rate - safety-first).

Liquid Staking - haSUI (Haedal)

Status: BUILT but **beta / pending mainnet verification**
End-user commands: - `"stake 10 SUI to haSUI"` - `"unstake 5 haSUI"`

What you see: A staking picker card with Haedal's stats. On stake, you mint haSUI; on unstake, you redeem instantly. The preview shows the expected output.

Limitations: The underlying PTB for haSUI was hand-built and tested in unit-test fixtures, but a live mainnet dry-run has not been completed. The builder is marked as needing mainnet verification. Use this feature for demo purposes only until full verification is complete.

Tech: Direct PTB (Haedal has no SDK). The Guardian enforces the same provider-specific action-shape gate as afSUI - a haSUI PTB cannot pass an afSUI-declared shape (and vice-versa).

Yield Advisor (Read-Only)

Status: WORKS - read-only recommendation engine
End-user commands: - `"what should I do with my USDC"` - `"where's the best yield"` - `"yield advice"` - `"yield options"`

What you see: A ranked recommendation card showing the best venues for your idle balances. The card lists stablecoin lending rates, top-TVL protocols, and liquid staking APYs - all live from on-chain sources. Each recommendation has an action button (e.g., "Lend 100 USDC on NAVI"); click to execute the normal Guardian-gated action flow.

What it is NOT: The advisor does not auto-execute; it does not fabricate numbers (a venue with no readable APY is omitted); it does not route your assets without your explicit sign. It is a **read-only research tool**.

Tech: Composed from existing read tools (portfolio, protocol registry, live APY reads) into a single ranked card.

Activity History (Read-Only)

Status: WORKS - immutable action log
End-user commands: - `"show my activity"` - `"my history"` - `"show my receipts"`

What you see: A reverse-chronological feed of your recent actions - swaps, transfers, lending moves, liquid staking, and **blocks** (deliberately refused intents). Each row shows the action type, the amount, the protocol, and the timestamp.

What is NOT shown: Profit/loss columns. The receipt schema stores no entry-USD baseline and pricing is spot-only, so cost-basis is undeterminable. A fabricated P&L would violate safety - amounts shown are values recorded at action time, not profit/loss.

Tech: Sourced from the memwal "action log" - immutable and **long-lived** (unlike a local browser history). BLOCK receipts are included as proof the firewall fired.

Multi-Step Chaining (Sequential)

Status: WIRED end-to-end - needs manual mainnet verification
End-user commands: - `"swap 5 SUI to USDC then lend it on NAVI"` - `"swap 10 SUI to USDC r■i deposit nó vào NAVI"` (Vietnamese) - `"swap 50 USDC to ETH and then borrow 10 ETH on NAVI"`

What you see: A plan card showing each step in the chain. Step 1 executes normally; when you sign, step 2 waits for the on-chain result of step 1 and resolves its amount from the **delta** (what step 1 produced), not your current wallet balance. The card streams live status as each step completes.

Limitations: - A page refresh loses an in-flight chain (durable resume across sessions is not yet implemented - you would restart the chain). - A transient stale-object error auto-rebuilds the step with fresh balances (you just re-confirm). - For `swap -> lend`, prefer the one-signature atomic mode below.

Tech: Sequential steps; each remains a normal single-action PTB through the Guardian. The delta resolver ensures step 2 consumes step 1's output (not your pre-existing balance). Each step is counted once toward daily spend (recycled values are not double-counted).

Atomic Composite Chaining (Single-Sign) - LIVE

Status: LIVE on mainnet (recipe `swap_lend_v1`)
End-user commands: - `"swap 2 SUI to USDC then lend it on navi"` -> on the plan card, click **"Run as 1 transaction (atomic)"**.

What you see: The two-step plan card offers a **"Run as 1 transaction (atomic)"** toggle. Click it and both legs are composed into ONE PTB you sign **once** - all-or-nothing. The tx-preview shows the full flow map: **You -> Cetus Aggregator (Swap -> ≈ estimated USDC) -> NAVI (deposit)**, with real protocol logos and the estimated output on each node.

What it is NOT: It does not change the safety model - the Guardian's `checkCompositeRecipe` re-verifies the entire composed PTB before the single signature. If the route can't be composed or any check fails, it **degrades to the sequential chain** above; funds and every Guardian check are unaffected. A SUI/gas shortfall is reported plainly ("not enough SUI…"), not as a route error.

Tech: Built via the Cetus aggregator's `routerSwap` (the swap-output coin is fed structurally into the NAVI deposit), with an upfront SUI-coverage gate. The Guardian gate enforces four invariants: (a) closed-recipe registry (no ad-hoc composition), (b) target multiset (exact PTB shape), (c) coin-type linkage (swap output = lend input), (d) delta/owner anti-leak + dual caps (USD + net-SUI). One signature, WYSIWYS, all-or-nothing. Full detail: `atomic-composite-mode.md`.

DeepBook Limit Order (POST_ONLY)

Status: WORKS
End-user commands: - `"limit order 10 SUI at 0.15 USDC"` - `"post-only limit order 100 USDC at 50 USDC per SUI"`

What you see: A limit order card showing the order price, order type (POST_ONLY - wait at your price, never market-buy), and estimated fill probability. The preview shows your BalanceManager position (the collateral for resting orders).

Tech: POST_ONLY orders are enforced on-chain; self-match is guarded; orders expire after a configurable window; the BalanceManager-ceiling gate prevents over-commitment.

Portfolio & Balances

Status: WORKS
End-user commands: - `"show my portfolio"` - `"what do I own"` - `"my balances"`

What you see: Live balances across all your tokens in USD value. Each token has a quick-action button (swap, send) that pre-fills the token in a form card.

Tech: Real-time on-chain balance read via the Sui RPC.

Cross-Chain Inflow (Wormhole Redeem)

Status: WORKS
End-user commands: - `"redeem my Wormhole VAA"` - `"complete cross-chain transfer"`

What you see: A redeem card showing the incoming asset, amount, and source chain. The Guardian verifies the VAA (Wormhole attestation), confirms the recipient is your wallet, and checks the fee model.

Tech: Wormhole Sui-side redeem, built SDK-free behind 9 fail-closed bridge gates.

How the Guardian Protects You

Every action goes through the **Guardian** - a deterministic, code-authoritative firewall. When you hit "Confirm," the Guardian:

1. **Re-derives the math independently** - it does not trust the LLM or any external data.
2. **Dry-runs the exact transaction bytes** to verify effects on-chain.
3. **Blocks on any failure** - a missing price, bad cap config, dry-run failure, or unknown target all block before a signature is requested.
4. **Binds your signature to the verified bytes** - the wallet signs exactly what the Guardian checked (WYSIWYS).

Result: An unsafe transaction never reaches your wallet. A BLOCK is written as an immutable receipt (proof the firewall worked), not just a failure message.

Frequently Asked Questions

Q: Can the copilot move my assets without my signature?
A: No. Every transaction requires your explicit wallet signature. The server builds unsigned PTBs only; keys

never leave your wallet.

Q: What if the copilot misunderstands my intent?
A: If the intent is ambiguous (e.g., "swap" without specifying an output token), the copilot renders an interactive form card instead of guessing. Fill it in and re-submit.

Q: What if I see a "BLOCK" message?
A: The Guardian refused an unsafe action. The card will explain why (e.g., "homoglyph lookalike", "health factor too low", "bad rate"). The refusal is immutable proof the firewall worked.

Q: Can I trust the preview amounts?
A: Yes. The preview is computed by the Guardian from the dry-run result - not the LLM. The amounts shown are real.

Q: What happens if my internet drops mid-chain?
A: If a multi-step chain is in-flight and you refresh the page, the in-flight chain state is lost. You would restart the chain. Durable cross-session resume is planned for a future update.

Q: How do I save a friend's address?
A: Click the Friend List button in the chat header and add a contact by name + address. The list is encrypted and stored per wallet.

Q: Do you store my conversations?
A: Yes, but encrypted. Your chat history is encrypted client-side with your wallet key - the server stores only opaque ciphertext. Only you can decrypt it (via wallet signature).

Safety Summary

- **Fail-closed** - unsafe transactions are blocked, never silently accepted.
- **WYSIWYS** - you sign the literal bytes the Guardian verified.
- **Zero server keys** - your funds never touch a server-held key.
- **Immutable receipts** - every action (and block) is recorded on Walrus for audit.
- **Privacy-first** - conversations are encrypted; personal data is not stored.

Every transaction is sealed before you sign. That's the Dewlock guarantee.